PRIORITIES — the ordered work queue

Opened 2026-08-04 from the overnight session. Every item traces to a measurement or a probe. Where an item has no number, it says so.

For *what a set is* and *which slots are genuinely open*, see BACKLOG.md — that file is domain analysis. This file is the queue.

How this queue is worked (standing mode)

Will, 2026-08-04: *"Keep allocating jobs to agents as jobs finish up and report their findings. Then take their findings and create future jobs for future agents to do. Eventually we will run out of bugs and jobs."*

The loop, and it is a loop rather than a plan because every pass changes the queue:

1. An agent reports. **Land its work first** — commit and push before dispatching anything new, or a crash loses it. Two agents this session had uncommitted work sitting on disk when they finished.
2. **Extract every finding, including the ones that were not the job.** The most valuable items in this file were incidental: the bring enumerator inventing brings, `applyMegaWeather` never firing, three ratchets red before the session began. Nobody was asked to find any of them.
3. **Add them here with their evidence and their owner**, in the priority band the harm justifies — not the band that is convenient.
4. **Dispatch into the free slot**, checking `FreePhysicalMemory` first. Never hand a division a file another live agent owns; say explicitly in the brief which files are taken.
5. Repeat.

When are we actually done

Not when the queue is empty — an empty queue means nobody has looked lately. Three real conditions:

- **The census stops moving.** It went 42/54 → 100/142 in one night. When a fresh block of probes turns up almost nothing red, that is evidence.
- **The differential holds at its floor across seeds.** It is at 1/400 at seed 20260804. One seed is one sample; a residual that stays flat across several is a residual.
- **Every published number has an artifact and a stamp**, so the P1 band cannot refill from prose.

The honest caveat, because this is where a project like this fools itself

A falling find-rate is not evidence of correctness. Tonight, 88 probes produced 40 red — a 45% hit rate on mechanics nobody had checked. If the next 88 produce 5, that is signal. If the next 88 are never written, the number simply stops moving and looks identical from the outside.

Every check in this repo says something about **the part of the engine it was pointed at.** The 2026-07-28 failures were all invisible to every automated check that existed, and the differential residual sat at "6/120" for a day while being **unreproducible** — two runs on identical source gave 6 and then 3. So the terminal state is not "no bugs found". It is **"the instruments are good enough that finding nothing means something"**, and we are not there yet.

How this is ordered

Not by size, and not by how annoying it is. By **what a wrong answer costs**, which here runs one way:

```
a broken MECHANIC  →  every rollout is wrong
  → every leaf value is wrong
    → every search decision is wrong
      → every H2H comparing two of them cancels the error and reports success
```

Fix what is FALSE, then what is UNCHECKABLE, then what is merely WEAK. An improvement measured on a broken engine is not an improvement, it is a new unknown.

Three standing rules apply throughout. Probe first and watch it fail. The census never goes down. A red test is fixed in the session that sees it red, or waived by Will by name — never filed.

P0 — FALSE. The engine reports things that are not the game.

Every rollout, leaf value and H2H ever run inherited all of this.

#	Item	Evidence	Owner		
0	**THE LEAF'S WEATHER STRING HAS NEVER MEANT ANYTHING TO THE ENGINE — take this before everything**	`board.weather` holds Showdown's `\`	-weather\	`line, which is a **move name**: sunnyday, raindance, sandstorm, snowscape.` medicham2-browser.js `compares against sun / rain / sand / snow (:464, :486-487, :934). Measured — Charizard Flamethrower into Garchomp: weather="sun" **92-109**, weather="sunnyday" **61-72**, weather="" **61-72**. **Truthy enough to suppress a guard, meaningless to every formula.** 0 of 9,040 HEAD playouts began in a weather MEDICHAM could read, while **5,320 carried a string** and 152 of 250 boards (60.8%) have one. **Fixing it moves ~65% of in-game leaf values** — an order of magnitude more than #37. **NARROWED 2026-08-04 — it is ONE BOUNDARY, not the engine.** Three paths set weather and only one is broken: (a) an **ability** on switch-in — weatherSetter` `emits the engine's own vocabulary directly (`drizzle {weather:"rain"}` `3,019 uses, sandstream` `1,659, snowwarning` `1,518, drought 884), read at medicham2:837 / :1090` —	ENGINE

#	Item	Evidence	Owner	
				`**works**; (b) a weather **move played inside the playout**, translated by` SD2WEATHER `at` :1481 `—` `**works**; (c) the **observed board handed in at the leaf boundary**,` applyField `in` rollout_leaf.js:321 `—` `**broken**, because` SD2WEATHER `lives in` medicham2-browser.js `and is not reachable from` rollout_leaf.js `. So the engine can SET weather correctly and cannot READ the weather already on the board it was given. **The fix is one translation call at one boundary**; the impact is unchanged. Filed as` docs/SEARCH.md` 5b
0a	**Sequencing: do NOT re-run any gate before #0 lands**	R1 and the explore sweep, R3, R4 and the in-game half of the leaf calibration are all invalidated by #0. Re-running them first buys a **second round** of invalidation. R2 is unaffected — leaf cost measured unchanged, 17.53 → 16.93 ms	—	
0b	`--miltank` **with** `--policy random` **searches NOTHING and looks normal**	All 81 `chooseMove` calls bail **silently** with 0 leaf calls (`acts=0, i=-1, board=false`); the only output is a preview-fallback line. **An R4 arm can run a whole job never having searched.** Count leaf calls before trusting any run	SEARCH / MEASURE	
1	~~ `redirects` — the attack VANISHES~~ **WITHDRAWN 2026-08-04. THE PROBE WAS WRONG.**	It aimed **Dragon** Claw at **Whimsicott** — Grass/**Fairy**, immune to Dragon. Follow Me fired correctly, pulled the attack off Incineroar, and landed it on a body that takes exactly zero; both arms read 0 and the verdict was written from that. Re-staged with Milotic, the same **unmodified** code reads `aimed 0 / redirector 101` . **Redirection works and always has. Nothing was invalidated.** The real gap was next door — `redirectsType` (Lightning Rod / Storm Drain, **1,901 uses**), where the engine only looked for	ENGINE → done	

#	Item	Evidence	Owner
		the Follow Me volatile. **Fixed.** Ninth probe to be wrong before the engine was, and the first that got believed	
2	`drain` **heals nothing**	8,553 uses. `dealt 51 to the foe; user 85 → 85 hp`	ENGINE *(landing)*
3	`choiceLock` **— two engines disagree about a fact**	5,886 uses. `board.js` passes `test-choice-lock.js` ; medicham2 does not. FACTS ARE GLOBAL, broken	ENGINE *(landing)*
4	`multiHit` **priced as one hit**	4,655 uses. The differential structurally cannot see this, so the probe is the only guard	ENGINE *(landing)*
5	`blocksSoundMoves` / `punishesContact`	2,726 / 1,761 uses	ENGINE
6	`fixedDamage` — **Seismic Toss worth zero**	1,122 uses, `mv.bp=0`	ENGINE *(landing)*
7	**Freeze-Dry**	1,252 uses	ENGINE
8	**Foul Play swings with the wrong Attack**	734 uses, 1.55× high. Body Press and Psyshock work *by accident*	ENGINE *(landing)*
9	**Disguise, Bulletproof, Soundproof, Overcoat, Dry Skin's Fire half**	the last 4 of 400 seeded differentials	ENGINE *(landing)*
10	`forcesSwitch` **(513),** `hazard` **(195)**	probed red	ENGINE
11	**39 tags still unprobed**	coverage 137/176	ENGINE
12	**12 census mechanics missing**	Facade, Ice Scales, Feint, recharge, Avalanche, Gyro Ball, Lightning Rod, Unnerve, Cursed Body, White Herb, Belly Drum, Tri Attack	ENGINE
13a	`benchRisk` **leaks the opponent's actual bring** — CONFIRMED, and SMALLER than feared	`fit_policy.js:384` hands `g.brought` (a scan of the **whole replay**) to the FOE's party, so at turn 1 `bench(foe)` is exactly the two of four they really chose, before either is seen. **19.99% of decisions** differ, always *narrowing* — the fit is more confident than the player was. **But measured, not argued: 0 of 58 weights move beyond 1.96 SE, held-out logL moves 0.00009, an order of magnitude inside a split-half floor of 3.9-4.3 SE.** "Every weight is corrupted" was my claim and it is FALSE, with a number. Worth fixing for correctness; does not on its own justify a refit	MEASURE
13	`train_value.py` **drops every forme-**	21.7% of faints, 22.7% of damaging events, 20.8% of all	MEASURE

#	Item	Evidence	Owner
	changed body	damage — silently. 21,196 of 21,629 dropped targets are megas. The `venusaurmega` / `venusaur-mega` bug in a new file; fix by calling `engine/mc_key.js`, not a second resolver	
13b	**THE FIT NEVER PASSED ABILITY OR MOVES TO THE BOARD**	`fit_policy.js:376` calls `setSheet(side, sp, {nature, item})`; `magnemite.js:522` calls it with `{nature, item, ability, moves}`. So **every board mon in the fit carries** `ability: ''` **and** `moves: []` while the bot that plays reads both. **51.52% of decisions** (~114,000 of 220,613) have a different vector, and **9 of 58 weights move beyond 1.96 published SE** — `clickCost` −0.258 → −0.795 (**−7.86 SE**), `diesBeforeMoving` −5.75, `switchSurvives1` +5.31. This is CLAUDE.md's named failure — *fitted WITH the sheet visible, played WITHOUT it* — **running in reverse**. Same two-field call in `joint_rows.js:95`, `branch_recall.js:68`, `ko_calibration.js:65`, `redirect_audit.js:120`, `surprise.js:79` and four more	MEASURE
13c	**The two fixes are COUPLED — landing 13b alone creates a new bug on 81% of the corpus**	`fit_policy.js:404` sets `mon.species` to the mega forme, after which `effective()` returns early at `board.js:2145` and never reaches the `isMega` ability branch. Inert today *only because* `mon.ability` is `''`. Fix `setSheet` alone and Charizard-Mega-X silently reverts Tough Claws → Blaze on **81.02% of recorded actions** (7,975 of 8,520 games). **Land 13b and 13c together or neither**	MEASURE
13d	`benchRisk` **is identically ZERO in every real MAG game**	`engine/magnemite.js` never calls `setParty` and never reads `bench()` — yet the feature carries a fitted weight of **−0.160 at z = −10.8**. A weight fitted at ten standard errors on a feature that is always zero in play. Only	MEASURE

#	Item	Evidence	Owner		
		`miltank.js:394-402` populates it, and with a *guess*			
13e	**The fit's items never go stale**	the store has no item event and `noteItem` is called only by `magnemite.js:574`, so offline a declared item stands forever. 11.77% of fit games contain an item-changing move (1,588 Knock Offs) and **5.85% of actions** occur after one landed. `PREFER OBSERVED OVER DECLARED` holds live and does not hold in the fit	MEASURE		
37	~~ `applyMegaWeather` has NEVER fired~~ **FIXED 2026-08-04, parity-verified**	both leaves call it and then assign `S.field.weather = f.weather \	\	"` **unconditionally one line later**. Mega Charizard Y has stood in **clear weather in every mid-battle rollout ever run** — ~26% of format usage. Deliberately left unfixed by SEARCH so it would not confound the leaf unification while ENGINE was mid-flight. **It is now unblocked and it is a Po**	ENGINE
38	**The 53.22% preview calibration figure describes a DELETED implementation**	MILTANK's preview no longer uses the hand-rolled greedy loop it was measured on. The figure must not be quoted for the shipped preview, and the MEASURE-vs-SEARCH contrast built on it (53.22% vs 50.99%) is void until re-measured **after the release boundary**	MEASURE		
39	~~The live budget may be 21 decisions~~ **SETTLED: 24 decisions — and it does NOT bind in testing, which is why nobody saw it**	per *decision* MILTANK sits comfortably inside 45 s, but a 7-minute chess clock is **420 s for the whole game**, so `budgetMs: 20000` is **21 decisions before the clock is gone**. This rests on whether the 45 s is drawn from the 420 s total — **an unverified rules assumption**, flagged rather than acted on. Verify against the actual VGC ruleset before anyone tunes `budgetMs`	SEARCH		

Blocked on a signature change: `writesAccuracy` / `accuracyMod` — `moveAccuracy(id, field)` takes neither attacker nor defender. **Possibly not bugs:** `critRatioUp` / `alwaysCrit` — `dmgRange` models no crit anywhere, so there is nothing to modify.

THE REFIT IS AUTHORISED — conditionally, and the condition is the point

Will, 2026-08-04, verbatim: *"I give permission to measure to start the refit once everything else above it has been cleared."*

Recorded here rather than in a chat log because a conditional permission that outlives the memory of its condition becomes an unconditional one. MEASURE **may** start the refit for items 13a/13b/13c. MEASURE **may not** start it until every box below is ticked, and MEASURE ticks them itself — nobody should have to adjudicate this at 5am.

Will reinforced the condition after three more findings landed, 2026-08-04: *"Make sure we address all findings before we start the refit tho."* So gate 1 is **every P0 and P1 item**, not the 1-12 that existed when this was first written. An item is cleared when it is **fixed**, or **closed with a verdict** (like #23), or **waived by Will by name** — never by being old.

Go / no-go, in order. Any NO means stop.

1. **Every P0 and P1 item is fixed, closed with a verdict, or waived by Will by name.** Including the ones filed after this gate was written: **#37** `applyMegaWeather`, **#23b replay publication**, **#23c the reconnect burst**, and **#38 the deleted-implementation figure**. Plus: `node engine/status.js` shows the census at or above **90 live** (it must never go down) and the seeded differential at the canonical seed **20260804** at or below tonight's **4/400**.
2. `node tests/run-all.js` is **fully green**. Not "green except". A red test is fixed or waived by Will by name — that rule is not suspended by this permission.
3. `node engine/artifact_audit.js` and `node engine/provenance.js --strict` both exit 0.
4. **13c is in the same working tree as 13b.** Landing the `setSheet` fix without the `effective()` fix puts a wrong ability on **81.02% of recorded actions**. If only one of the two is ready, the answer is NO.
5. `engine/feature_fixture.js --check` has been run and its result **recorded before the refit**, so there is a stated before-state. It will legitimately FAIL after 13b/13c — that is the whole point, the feature function is changing — but the failure has to be the expected one and not a surprise.

What the refit is NOT gated on, and why — this corrects an assumption I made earlier. The engine fixes in P0 1-12 land in `engine/medicham2-browser.js` and `data/abra-tags.js`. The fit's features come from `engine/board.js` via `engine/fit_policy.js`, which is a **separate implementation** — that separateness is itself defect #3, but it means a medicham2 fix does **not** by itself invalidate the fit. `status.js` already reports this correctly: the refit edge is judged by `feature_fixture --check` hashing all 58 columns, not by an engine mtime. So the refit is triggered by **13a/13b/13c touching the feature path**, not by the engine release.

The engine release boundary (P0.5, below) still gates every **SEARCH** run and every H2H. Two different boundaries, two different reasons; do not collapse them.

After the refit, in the same pass: the seven restamps, `node engine/status.js --write`, and every number that quoted the old weights. A refit that lands without them is how a version drifts.

WOBBUFFET RE-RUN — RAN 2026-08-04, VOID, STILL OPEN

Will, 2026-08-04: *"rerun wobba" … "yes do the search once engine is all wrapped up."*

> ***It ran at full size and the tree moved under it.*** `data/policy-weights.json` *— the defender — was* ***refitted at 22:15:24 UTC, mid-run****;* `engine/board.js` *was written around round 5; and* `engine/medicham2-browser.js` *changed content between 22:29:04 and 22:30:34 and again at 22:31, sampled with* `run_stamp.sourceDigests()`. ***The engine was not wrapped.*** *The old figure is retracted anyway, so the position is now no exploitability number at all. Two findings survive because they are about the tool: the hill-climb stalls in the shipped dimension (its step count is withheld with the void run, withdrawn 2026-09-11), and* `provenance.js` ***falsely cleared*** *the new artifact on an mtime test it passes by 153 s while having read its input 34 minutes before that input was rewritten. Full timeline, defects and*

> *the prepared re-run:* `docs/SEARCH.md` *§R8.* ***The sequencing rule below was right and was not actually satisfied — verify the digests are still, do not take "wrapped" on report.***

`data/exploitability.json` was **measured on 17 features against the 58 we ship** — three feature generations stale, on a policy we have since made **more deterministic** by shipping greedy-over-sampling. Its share and interval are **QUARANTINED — withheld, not annotated**: the artifact is downstream of MEDICHAM, and it becomes quotable again when the gate opens AND this is re-run: `node engine/exploit.js`. `provenance.js --strict` exits 1 on it and `tests/run-all.js` gates on that, so it is a red test, not a nicety. The Stadium already renders it as NOT MEASURED with the old figure struck through, so the page is honest either way; this is about the artifact.

Why it is worth re-running rather than retiring. It is the only measurement in this project that is not bots grading bots on average — it grades **readability**. `docs/MODELS.md` records the rule: *a policy can improve on average and stay exactly as exploitable; those are different numbers and only one of them has been moving*. We have just spent a day improving on average.

The sequencing is the point, and Will set it. Run it **after** the engine is wrapped and the release is cut — not before. An exploitability search plays games through the engine, so a run started while ENGINE is still landing mechanics is born `PRE-CHANGE` and buys a second re-run. That is the mistake this file exists to prevent, and it would be embarrassing to make it on the item that gates the suite.

Before starting it: record the engine digests, and abort and report if they move mid-run — the leaf calibration did exactly this and it is why that result survived a moving tree. Command is `engine/exploit.js --target <weights.json>`; it can also be pointed at DODUO, which is the only way to test the exploitability argument the 42.0% result explicitly does **not** settle.

P0.5 — the release boundary. Do this the moment P0 lands.

Cut a named, frozen engine release. Every SEARCH baseline and every MEASURE number currently describes an engine in which redirection deleted attacks. Cutting the release triggers the refit and the seven restamps, and it is the only thing that stops the next result being born `PRE-CHANGE`. Until then, **start no wide run** — it will measure a build that stops existing. *(DIVISIONS.md rule 1.)*

P1 — UNCHECKABLE. Numbers we quote that nothing can reproduce.

#	Item	Evidence	Owner
14	**R2 is re-run or it is nothing**	base timings are never dumped, and a duration is not recomputable in principle — no CPU, node version or load recorded. It also timed `explore=0/maxTurns=20` when the shipped leaf is `1.0/60`	MEASURE
15	**17 twin-test statistics on the site with no artifact**	`web/index.html:1107-1108` — `52.3% of decisive pairs (95% CI 50.9–53.6)`, `87,150 games`, `43,575 paired`, stamped "Confirmed 2026-07-30". Only `586,816` is backed. Either MEASURE writes the artifact or the paragraphs go	MEASURE + WEB
16a	**RESOLVED — 8,414 vs 6,943 is not a bug**	Different populations, both current. **6,943** = clean games in `games.ladder.jsonl` only, **no sheet requirement**. **8,414** = clean **open-sheet** games across **three** stores — `fit_policy.js:272` iterates bo3 + ots + ladder and `:250` rejects anything without both sheets. The 2026-07-31 split was bo3 3,807 (54.7%) + ots 2,891 (41.5%) + ladder **268 (3.8%)**, because the ladder format's Open Team Sheets are optional and only ~1% of it carries one. **Never print them side by side without their population**	closed

#	Item	Evidence	Owner
16	**Two live definitions of "clean games"**	`live.js` / `winrate-backtest.json` say 6,943; `meta-usage.json` / `roles-eval.json` / `guru-matchups.json` say ~5,269. The front door renders the larger beside results computed on the smaller. **OPS diagnosed the cause: pure staleness, not two definitions** — both are `quality.js` / `quality.py` reading `quality-filter.json`, pinned identical by `tests/test-quality.js`. The funnel was computed at 29,117 collected; the store is now 38,587. `node engine/analyze.js data/games.ladder.jsonl` closes it	MEASURE
17	`guru.js` **says** `n_decisive: 0`, `guru-matchups.json` **says** `6`	and `guru.js` is *generated from* it. The site is self-consistent only by luck of which file it reads	MEASURE
18	**Exploitability is NOT MEASURED — worse than stale**	WOBBUFFET's earlier 17-feature figure is retracted (2026-08-04) and withheld; the 58-feature re-run is **void** (defender refitted mid-run, simulator moved twice more) and its search stalled — the step count is withheld with it (withdrawn 2026-09-11). `exploit.js` needs a run stamp and a dimension-aware step rule before the next attempt. `docs/SEARCH.md` §R8	MEASURE
18a	`exploit.js` **stamps nothing at all**	no engine digests, no digest of the target vector it read, no node version, no pool size, no `n_measured` / `n_unit`. This is why a mid-run refit of its own defender was invisible. Every other gate here carries a stamp	MEASURE
18b	`provenance.js` **cleared an artifact computed from a superseded input**	mtime test only: `exploitability.json` (22:17:57) is newer than `policy-weights.json` (22:15:24) and passes, but the process read the weights at 21:41. A generator must stamp the **content digest** of each input at read time and provenance must compare digests. `run_stamp.sourceDigests()` already does this for leaf sources	MEASURE
19	`rollout_r1_join.py` **writes naked** `isoformat()`	`2026-08-03T04:14:10` parses in JS as `08:14:10Z` — a four-hour shift. Latent only because status.js refuses withdrawn artifacts	MEASURE
20	`n_measured` / `n_unit` **missing on R1 and R4**	one line each	MEASURE
21	**PORY tooltip contradicts the PORY room**	`index.html:2149` "beats a coin, well calibrated" vs `:915` "I add nothing over counting"	WEB
22	**WEB has no number of its own**	the fraction of rendered figures tracing to an artifact is unmeasured. The division that polices drift does not get to exempt itself	WEB
23	~~`data/games.ots.jsonl` not written since July~~ **CLOSED 2026-08-04**	**It is a completed external import, not a stalled collector.** The hourly Action pulls exactly two formats; the Force-Open-Team-Sheets one (`gen9championsvgc2026regmbbo3`) lands in `data/games.bo3.jsonl`, which is the most recently written store on disk. OTS collection never stopped — it never lived in `games.ots.jsonl`. `status.js` printed the frozen archive beside the live store and omitted the collecting one, which is what read as "collection stopped"; **fixed**	OPS → done
23b	**Replay publication is failing and nothing reports it**	`data/live-games/` holds **30** recordings with real Showdown ids; `data/live-games/replays.txt` holds **2** URLs. The two published are the 2nd and 3rd oldest games, so the path worked once and has not since. This directly contradicts `docs/OPS.md:54` — *"Every OTS game recorded with its replay published"*. Cause NOT established: candidates are `/leave` at `mag_bot.js:1015` firing in the same handler pass as `/savereplay` at `:827`, or rooms expiring server-side before the login sweep reaches them. Needs a live run with raw server output — **and must not be run while a battle is live**	OPS diagnosis → OPS/live
23c	**The login-time replay sweep is a throttle hazard that grows**	`mag_bot.js:600-617` scans for every unpublished recording and fires `/join` + `/savereplay` for each, 4 s after login — currently **28 rooms**	OPS/live

#	Item	Evidence	Owner
		= 56 commands in one burst, on every reconnect, with no rate limit, growing with every unpublished game. Fixing 23b disarms it	
23d	**The 18% is 89.5% bots, and non-OTS is not a store rule at all**	named bots **−19,104 (80.1% of all discards)**, behavioural bots −2,247 (9.4%), incomplete bring −1,932 (8.1%), stubs −532 (2.2%), forfeits −33 (0.14%). Duplicates never reach the funnel — `quality.js:89-103` dedupes first. **Caveat that must travel with these:** the funnel is order-dependent, not an exclusive attribution — a bot game that was also a stub is charged entirely to the bot rule	closed as understood

P2 — WEAK. Real components that under-perform.

The leaf gates most of this. Advantage-reweighted BC needs a critic worth trusting; so does downside-aware selection. Neither starts before the leaf question is settled.

#	Item	Why	Owner
24	**The action-ranking backtest**	Every leaf number so far scores a *position*. A search leaf ranks *actions*, and two leaves with identical Brier can order actions completely differently. **Nothing measured so far touches this.** Enumerate expressible joint actions per decision, score with `rolloutWinProb` and `materialP`, report rank-correlation and argmax-agreement **with intervals** — not a winner. ~2,000 decisions ≈ 5 h at n=200, ~1 h with a 40-rollout screen. Do NOT size it as a superiority test; that needs ~190,000 decisions	SEARCH
25	**MILTANK ships TWO leaves**	`miltank.js:216-222` is a second hand-rolled playout loop that never calls `rolloutWinProb`. The 53.22%-vs-50.99% contrast is partly a contrast between *leaves*, not settings	SEARCH
26	**Preview calls `battleInit({seeded:true})` on a fresh game**	suppresses turn-1 Intimidate and weather setters	SEARCH
27	**`board.js` scores every switch with one flat feature**	MAG's switching measured **10 points worse than never switching**, and a switch was on the menu on 17 of 121 R3 decisions. A hole in the live policy, not a measurement problem	MEASURE *(owns the refit)*
28	**Downside-aware selection**	MILTANK argmaxes the **mean** rollout value, so it cannot tell a flat 60% line from a 90% line that loses on the spot. The distribution is already computed and everything past the first moment is thrown away. Selection rule only — no new model, no refit, scorable on the R4 harness. From a domain expert describing how strong players actually decide	SEARCH
29	**MACHAMP re-run**	half-run on a 17-feature vector against today's 56, and the only component whose objective is winning rather than imitating	MEASURE
30	**Fit the same LINEAR vector to a winning objective**	the cheap ablation separating "capacity" from "objective". If the null moves, capacity was never the binding constraint and architecture is second-order. We can afford this ablation; the external work does not have one	MEASURE
31	**Self-play volume 0.15:1 → 10:1**	the external pipeline that works runs 10 self-play games per human game; MEW has 1,000 against 6,890, built and gated. **After the release boundary**	MEASURE
32	**PPO clipping instead of the hand-rolled trust region**	`train_policy.js`, bounded change to one file	MEASURE
33	**`--miltank-explore` on `mew.js`**	two parsed flags, and the A/B that settles explore=1.0 as a *player* rather than a *judge* becomes a standard ~420-game paired SPRT. Currently not runnable at all	MEASURE

#	Item	Why	Owner
34	`train_policy.js` `writeWeights` **provenance lie**	copies the base file's `generated` / `source`, so a REINFORCE checkpoint claims `source: engine/fit_policy.js`	ENGINE
35	`battleResult` **cannot tell a wipeout from an expired clock**	real hazard, measured at 0.2–0.5% of playouts — a correctness fix, **not** the explanation for anything	ENGINE
36	**45-second live decision budget**	real VGC allows 45 s on a 7-minute clock. R2's leaf cost has never been checked against it	SEARCH

Capacity

8 physical / 16 logical cores (Ryzen 7 7735HS). **RAM is the ceiling, not cores.**

16 GB installed, 13.35 GB usable, and the gap is not recoverable from software. The Radeon 680M reserves ~2.65 GB as a UMA frame buffer in firmware. Only a BIOS change (UMA frame buffer 2 GB → 512 MB on this ThinkBook 14 G7 ARP) recovers it. Do not spend time looking for a Windows setting; there isn't one.

Measured breakdown at 04:17 on 2026-08-04, with four agents live and 3.6 GB free:

process	count	GB
`claude` — this session **plus every other Claude window**	20	2.88
`vmmem`	1	1.43
`node` — one agent's engine run	1	1.34
`svchost`	92	1.23
`msedgewebview2`	11	0.42

`vmmem` **is CONFIRMED as Virtualization-Based Security, not a leftover VM.** Checked directly: `VirtualizationBasedSecurityStatus = 2` (enabled and running) and `SecurityServicesRunning = 2` (Hypervisor-Enforced Code Integrity / Memory Integrity), with `HvHost` and `vmcompute` running. WSL is **not installed**; Docker and the Android subsystem are **not running**; there are no Hyper-V VMs to list. It started with the machine, not with any session.

It was suspected of being a Claude Cowork relic. It is not. **It is a security feature, it is a protected process that cannot be killed, and it should not be disabled to buy memory.** Recorded because it looks exactly like reclaimable overhead and is not — the next person to go looking for 1.4 GB will land on it too.

The `claude` **figure is ONE window, and this was got wrong once already.** 2.88 GB across 20 processes looks like several sessions and is not: it is the Electron shell and renderer (~1.1 GB between two processes), the agent workers (~200-350 MB each), and roughly fourteen MCP servers at 14-116 MB apiece. Closing windows is not the lever, because there is one window.

The only lever actually under our control is agent concurrency. Budget an agent at ~300 MB itself, plus up to ~1.3 GB if it spawns a `node` engine run. Four light agents cost about what one heavy one does, so the cap is not a count — it is how many are holding a rollout at the same time. Check `FreePhysicalMemory` before adding one rather than assuming a number.

`mew_farm` runs 12 procs at `--conc 1` (44–46 games/sec). `--conc` must stay at 1: the simulator is synchronous and CPU-bound, so in-process concurrency never overlaps real work and only multiplies GC pressure. Measured 8 procs at `--conc 4` = 11 games/sec against the same 8 at `--conc 1` = 38.

For agents: **4 concurrent is the working cap** while engine work is live, not the 6 in DIVISIONS.md. Two agents each holding a rollout is what actually exhausts memory.

Not on this list, deliberately

- **A transformer, or any architecture change.** The cheap ablation (#30) runs first. Changing architecture and objective at once is exactly the un-ablated result we criticised in others.
- **More MAG features.** Four measured nulls, with an overdispersion check (~1.00) saying they are genuine rather than a real effect hidden by team heterogeneity.
- **Deeper search.** Our evidence and the external evidence agree that the leaf bounds a search agent. A better search over a coin-flip evaluator is a better-organised coin flip.